

Explainable AI for Software Engineering

Chakkrit (Kla) Tantithamthavorn, Jirayus Jiarapakdee
Monash University, Melbourne, Australia.

Abstract—The success of software engineering projects largely depends on complex decision-making. For example, which tasks should a developer do first, who should perform this task, is the software of high quality, is a software system reliable and resilient enough to deploy, etc. However, erroneous decision-making for these complex questions is costly in terms of money and reputation. Thus, Artificial Intelligence/Machine Learning (AI/ML) techniques have been widely used in software engineering for developing software analytics tools and techniques to improve decision-making, developer productivity, and software quality. However, the predictions of such AI/ML models for software engineering are still not practical (i.e., coarse-grained), not explainable, and not actionable. These concerns often hinder the adoption of AI/ML models in software engineering practices. In addition, many recent studies still focus on improving the accuracy, while a few of them focus on improving explainability. Are we moving in the right direction? How can we better improve the SE community (both research and education)?

In this tutorial, we first provide a concise yet essential introduction to the most important aspects of Explainable AI and a hands-on tutorial of Explainable AI tools and techniques. Then, we introduce the fundamental knowledge of defect prediction (an example application of AI for Software Engineering). Finally, we demonstrate three successful case studies on how Explainable AI techniques can be used to address the aforementioned challenges by making the predictions of software defect prediction models more practical, explainable, and actionable. The materials are available at <https://xai4se.github.io>.

Index Terms—Explainable AI, Software Engineering

I. TUTORIAL OUTLINE

Our 1.5-hour tutorial consists of three parts:

- Part 1-Explainable AI: We first provide a concise yet essential introduction to the most important aspects of Explainable AI and a hands-on tutorial of Explainable AI tools and techniques.
- Part 2-Defect Prediction Models: We introduce the fundamental knowledge of defect prediction (an example application of AI for Software Engineering)
- Part 3-Explainable AI for Software Engineering: We demonstrate three successful case studies on how Explainable AI techniques can be used to address the aforementioned challenges by making the predictions of software defect prediction models more practical, explainable, and actionable.

Part 1: Introduction to Explainable AI

We first provide a concise yet essential introduction to the most important aspects of Explainable AI. At the beginning of the tutorial, we will introduce the motivation, definitions, and concept of Explainable AI. We will also discuss a theory of explanations (e.g., what are the explainability goals, what are

the explanations, what are the types of intelligibility questions, what are the scopes of explanations, what are the types of explanations). We will also discuss the model-specific and model-agnostic tools and techniques that are extensively-used in the Explainable AI community (e.g., LIME [9], SHAP [5]) for explaining the predictions of AI/ML models with a hands-on tutorial.

Part 2: Defect Prediction Models

In this part, we will first introduce the fundamental knowledge of defect prediction technologies. This include five key steps (1) Data Collection, (2) Data Preprocessing, (3) Model Construction, (4) Model Evaluation, and (5) Model Ranking, along with the following empirical-grounded guidelines [10]. We describe details below.

(Step 1) Data Collection. Poor quality or noisy defect datasets could lead to inaccurate predictions and insights. We found that techniques for generating ground-truth data is often not accurate, impacting the quality of defect datasets. Thus, we will discuss how to: consider affected releases (i.e., the actual software releases that are affected) to label whether a file is considered to be defective or clean, instead of the assumptions of a post-release window period (i.e., any defects that are fixed after 6 months) [17], while not to be concerned about issue report misclassification [12].

(Step 2) Data Analysis. Defect datasets are highly imbalanced with a defective ratio of $<10\%$. Defect models trained on class imbalance datasets often produce inaccurate models. Our TSE'19 paper suggested to consider using optimised SMOTE to improve the predictive accuracy, i.e., handling the class imbalance of the training datasets prior to training the defect models. Thus, we will discuss how to:

- Handle collinearity and multicollinearity when interpreting defect models (i.e., understanding what are the most important variables) [2].
- Use AutoSpearman for handling collinearity and multicollinearity [3], and avoid using existing automated feature selection techniques (e.g., Stepwise Regression) if the goal is to interpret defect prediction models, as they fail to mitigate correlated features and are dependent on random seeds [4].

(Step 3) Model Construction. There exist a large number of off-the-shelf classification techniques that can be used with a large number of possible combination of hyperparameter settings that can be configured. Sadly, practitioners often asked which techniques and which settings should be used. Thus, we will discuss how to:

- Explore various classification techniques and hyperparameter settings. Optimised random forest and optimised extreme gradient boosting trees often produce the most accurate defect prediction models [13].
- Always optimize the hyperparameter settings of defect prediction models (e.g., using Grid Search, Random Search, Genetic Algorithm, Differential Evolution) [15].
- Use the Scott-Knott ESD test to identify the best classification techniques that are statistically significantly different with non-negligible Cliff's $|\delta|$ effect size [14].
- Estimate the accuracy of defect prediction models using out-of-sample bootstrap model validation techniques when defect datasets are very small (i.e., EPV < 10) [14].
- Using ANOVA Type-II for explaining the defect prediction models (built using regression models), instead of using ANOVA Type-I [2].

Part 3-Explainable AI for Software Engineering Including Hands-on Examples

In this part, we will discuss three successful case studies on how Explainable AI techniques can be used to address the aforementioned challenges by making the predictions of software defect prediction models more practical [6, 16], explainable [1, 2], and actionable [7, 8, 11]. In particular, the hands-on demo will focus on how to explain defect prediction models in order to generate global explanations [2] and instance explanations [1].

II. TARGET PARTICIPANTS

The target participants of this tutorial include: **SE researchers and PhD students** who want to learn more about the intersection of Explainable AI and Software Engineering. **Software practitioners** who already use Python for as data science, machine learning, research, and analysis and wish to apply their data science knowledge to software data. **Software analysts and data scientists** who want to understand and avoid pitfalls when designing software analytics. **Project managers** who involve high-stakes decision-making and need software analytics to make smarter data-driven business decisions. There are no prerequisites, no audio-visual, and no pre-installed technical requirements for attending this tutorial.

III. INSTRUCTORS' BIOGRAPHY AND EXPERTISE

Dr. Chakkrit (Kla) Tantithamthavorn is a Senior Lecturer in Software Engineering and a 2020 ARC DECRA Fellow in the Faculty of Information Technology, Monash University, Australia. He is leading a new research area of Explainable AI for Software Engineering. His current fellowship is focusing on the development of Practical and Explainable Analytics to Prevent Future Software Defects. His work has been published at several top-tier software engineering venues, such as TSE, ICSE, EMSE, MSR, IST.

modelling, experimental design, and software engineering to

Dr. Jirayus Jiarapakdee is graduated from Monash University, Australia. His research interests include empirical software engineering and mining software repositories (MSR). The goal of his Ph.D. is to apply the knowledge of statistical

improve the explainability of defect prediction models. Contact him at jirayus.jiar@gmail.com.

Acknowledgement. Chakkrit Tantithamthavorn was supported by ARC DECRA Fellowship (DE200100941).

REFERENCES

- [1] J. Jiarapakdee, C. Tantithamthavorn, H. K. Dam, and J. Grundy, "An Empirical Study of Model-Agnostic Techniques for Defect Prediction Models," *IEEE Transactions on Software Engineering (TSE)*, p. To Appear, 2020.
- [2] J. Jiarapakdee, C. Tantithamthavorn, and A. E. Hassan, "The Impact of Correlated Metrics on Defect Models," *IEEE Transactions on Software Engineering (TSE)*, p. To Appear, 2019.
- [3] J. Jiarapakdee, C. Tantithamthavorn, and C. Treude, "AutoSpearman: Automatically Mitigating Correlated Software Metrics for Interpreting Defect Models," in *Proceedings of the International Conference on Software Maintenance and Evolution (ICSME)*, 2018, pp. 92–103.
- [4] —, "The impact of automated feature selection techniques on the interpretation of defect models," *Empirical Software Engineering*, vol. 25, no. 5, pp. 3590–3638, 2020.
- [5] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems (NIPS)*, 2017, pp. 4765–4774.
- [6] C. Pornprasit and C. Tantithamthavorn, "JITLine: A Simpler, Better, Faster, Finer-grained Just-In-Time Defect Prediction," in *Proceedings of the International Conference on Mining Software Repositories (MSR)*, 2021.
- [7] C. Pornprasit, C. Tantithamthavorn, J. Jiarapakdee, M. Fu, and P. Thongtanunam, "PyExplainer: Explaining the Predictions of Just-In-Time Defect Models," in *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2021.
- [8] D. Rajapaksha, C. Tantithamthavorn, J. Jiarapakdee, C. Bergmeir, J. Grundy, and W. Buntine, "SQLPlanner: Generating Data-Informed Software Quality Improvement Plans," *IEEE Transactions on Software Engineering (TSE)*, 2021.
- [9] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the Predictions of Any Classifier," in *Proceedings of the International Conference on Knowledge Discovery & Data Mining (KDD)*, 2016, pp. 1135–1144.
- [10] C. Tantithamthavorn and A. E. Hassan, "An Experience Report on Defect Modelling in Practice: Pitfalls and Challenges," in *In Proceedings of the International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*, 2018, pp. 286–295.
- [11] C. Tantithamthavorn, J. Jiarapakdee, and J. Grundy, "Actionable analytics: Stop telling me what it is; please tell me what to do," *IEEE Software*, vol. 38, no. 4, pp. 115–120, 2021.
- [12] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, A. Ihara, and K. Matsumoto, "The Impact of Mislabelling on the Performance and Interpretation of Defect Prediction Models," in *Proceeding of the International Conference on Software Engineering (ICSE)*, 2015, pp. 812–823.
- [13] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, and K. Matsumoto, "Automated Parameter Optimization of Classification Techniques for Defect Prediction Models," in *Proceedings of the International Conference on Software Engineering (ICSE)*, 2016, pp. 321–332.
- [14] —, "An Empirical Comparison of Model Validation Techniques for Defect Prediction Models," *IEEE Transactions on Software Engineering (TSE)*, vol. 43, no. 1, pp. 1–18, 2017.
- [15] —, "The Impact of Automated Parameter Optimization on Defect Prediction Models," *IEEE Transactions on Software Engineering (TSE)*, pp. 683–711, 2018.
- [16] S. Wattanakriengkrai, P. Thongtanunam, C. Tantithamthavorn, H. Hata, and K. Matsumoto, "Predicting defective lines using a model-agnostic technique," *IEEE Transactions on Software Engineering (TSE)*, 2020.
- [17] S. Yathish, J. Jiarapakdee, P. Thongtanunam, and C. Tantithamthavorn, "Mining Software Defects: Should We Consider Affected Releases?" in *In Proceedings of the International Conference on Software Engineering (ICSE)*, 2019, p. To Appear.